

PROJECT REPORT

“KERNEL MONITOR”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Section No.: 7 | Team No.: 17

By

CHALLAGULLA SHRAVYA 2520030322

GANDU RISHITHA 2520030101

M.V.L. SAMHITA 2520030509

Under the Esteemed Guidance of

Dr Hema Kolla

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Academic Year: 2026–27

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

We hereby declare that the project report entitled **“Kernel Monitor”** is a bonafide work carried out by **Challagulla Shravya (2520030322)**, **Gandu Rishitha (2520030101)**, and **M.V.L. Samhita (2520030509)** as part of the Operating Systems and Systems Programming course during the academic year 2026–27. The report describes the implementation, design, testing, and execution of a Linux-based monitoring application developed in C.

The project work presented in this report is based on the team implementation and the submitted project problem statement. Where external Linux utilities or interfaces are used, they are identified in the relevant sections.

G Rishitha 2520030101

MVL Samhita 2520030509

CH Shravya 2520030322

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **“Kernel Monitor”** is a bonafide work carried out by **Challagulla Shravya (2520030322)**, **Gandu Rishitha (2520030101)**, and **M.V.L. Samhita (2520030509)**, in partial fulfillment of the requirements for the Operating Systems and Systems Programming course during the academic year **2026–27**.

The project implements a Linux-based C monitoring application that reads live operating-system information, tracks multiple processes, calculates CPU and memory usage, observes process I/O, provides system-call activity for a selected process through strace, and generates threshold-based alerts.

**Signature of the Course
Coordinator**

Signature of the Guide

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty members of the Department of Computer Science and Engineering for their guidance and support during the development of the Kernel Monitor project. We are thankful for the opportunity to apply Operating Systems and System Programming concepts to a practical Linux-based application.

We also thank the course faculty and our classmates for their suggestions, discussions, and support during the implementation and testing stages. Finally, we thank our parents and friends for their encouragement and cooperation.

G Rishitha 2520030101

MVL Samhita 2520030509

CH Shravya 2520030322

Abstract

The Kernel Monitor is a Linux-based system monitoring application written in C that continuously observes running processes and important system resources. The implemented system discovers processes from the Linux `/proc` filesystem and displays process identifiers, names, states, CPU usage, memory usage, thread counts, CPU-core information, running time, and process I/O rates. The monitor calculates per-process CPU usage from process and system jiffy deltas and calculates memory usage using resident-set memory information. It also maintains CPU and memory analysis values such as current usage, average, peak, minimum, trend, and status.

A selected process can be monitored in greater detail. By default, the process with the highest measured CPU usage is selected, while a specific PID can also be supplied as a command-line argument. System-call activity for the selected process is collected using the external Linux `strace` utility. The monitor also detects new and terminated processes, maintains process-state counts, monitors read/write activity, and generates alerts when CPU or memory usage remains above 80% for three consecutive samples.

The application uses a one-second monitoring cycle and a terminal-based display with formatted sections. The project demonstrates practical Operating Systems concepts including process management, memory management, Linux virtual filesystems, system calls, signals, file I/O, resource monitoring, and system-level programming in C.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–2
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		3–4
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		5
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		6–7
		4.1 C Programming	
		4.2 GCC Compiler and Make	
		4.3 Linux System Calls / APIs	
		4.4 Linux /proc Filesystem	
		4.5 strace	
		4.6 Shell Utilities and Testing	

S. No.	Chapter	Topics / Subtopics	Page No.
5	System Architecture and Design		8–10
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		11–13
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C Implementation	
		6.4 Linux Interfaces / APIs Used	
		6.5 Process and Thread Monitoring	
		6.6 System Call Monitoring	
		6.7 Memory and I/O Monitoring	
		6.8 Alert and Signal Handling	
7	Execution and Demonstration		14–15
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Testing	
		7.5 Demonstration of OS Concepts	
8	Conclusion		16
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		17

S. No.	Chapter	Topics / Subtopics	Page No.
		9.1 Books	
		9.2 Research / Technical Background	
		9.3 Linux / GNU Documentation	
		9.4 Project Files Used	

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

Modern Linux systems execute many processes and background activities simultaneously. Each process consumes CPU time, memory, processor resources, and may perform file or device I/O. The project problem statement identifies the need for a Linux-based monitor that makes this activity easier to observe and helps identify resource-intensive or abnormal processes.

The implemented Kernel Monitor extends this idea into a live terminal application. Instead of monitoring only overall system values, it scans all numeric PID directories under `/proc`, maintains a process list, calculates per-process CPU and memory values, and sorts processes by CPU usage. The user can then inspect the selected process in more detail.

1.2 Problem Statement

Linux systems execute numerous processes and system operations at the same time. Basic system information may not be sufficient to quickly identify which process is consuming CPU, memory, or I/O resources. It may also be difficult to understand process states and system-call activity from isolated commands.

Therefore, a Linux-based Kernel Monitor is required to continuously collect process-level and system-level information, present it in a readable form, identify high resource usage, and provide additional activity information for a selected process.

1.3 Scope of the Project

- Monitor all currently visible Linux processes through `/proc`.
- Display PID, process name, parent PID, state, CPU usage, memory usage, thread count, CPU core, and running time.
- Calculate real per-process CPU usage using process and system jiffy deltas.
- Calculate resident memory usage using `VmRSS` and system memory totals.
- Calculate read/write I/O rates from `/proc/<PID>/io`.
- Track process creation and termination between monitoring cycles.
- Select a process automatically by highest CPU usage or explicitly using a PID.
- Capture system-call summaries and recent calls for the selected process using `strace`.
- Provide CPU/memory analysis and threshold-based alerts.
- Operate as a terminal application without a database or web backend.

1.4 Objectives of the Project

1. Monitor CPU, memory, and system resource utilization in real time.
2. Track running processes and their resource consumption.

3. Collect system-level information using Linux interfaces and system programming techniques.
4. Identify excessive resource usage and provide useful alerts and monitoring information.
5. Provide detailed information for a selected process, including system-call activity.
6. Demonstrate practical Operating Systems concepts using a working C implementation.

1.5 Importance of the Application

The project is useful as an educational system-monitoring tool because it connects theoretical OS concepts with live Linux kernel information. Students can observe how process states, CPU time, memory, I/O, and system calls change while programs are running. The terminal presentation also makes the application suitable for Linux/Ubuntu laboratory demonstrations.

1.6 Target Users / Audience

- Operating Systems and System Programming students.
- Students learning Linux process management and /proc interfaces.
- Developers who want a lightweight terminal-based process monitoring example.
- Faculty members demonstrating CPU, memory, process, I/O, and system-call concepts.

CHAPTER 2 — OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Concept	How it is demonstrated
Process Management	Scans /proc for PIDs, tracks process lifecycle, states, parent PID, threads, and CPU usage.
CPU Scheduling / CPU Time	Uses utime/stime jiffies and system jiffies to estimate process CPU percentage.
Memory Management	Reads VmRSS and system memory values to calculate resident memory consumption.
File Systems	Uses the virtual /proc filesystem and file operations to obtain kernel-maintained information.
System Calls	Uses Linux APIs such as open/read/close-related file operations and captures selected-process system calls through strace.
Signals	SIGINT and SIGTERM are handled for clean shutdown.
I/O Monitoring	Reads read_bytes and write_bytes from /proc/<PID>/io and calculates rates.
Threads	Reads /proc/<PID>/task and process thread information.

2.2 Existing Methods / Algorithms

Traditional Linux utilities such as top, ps, free, and similar commands provide individual views of process or system information. The project problem statement also notes that basic system information can make it difficult to identify resource-intensive and abnormal activity. The Kernel Monitor combines several measurements in one terminal dashboard and adds selected-process system-call activity.

The project does not implement a new CPU scheduling algorithm or memory-allocation algorithm. Instead, it uses kernel-provided measurements and performs calculations over consecutive samples.

2.3 Proposed Approach

1. Scan /proc and identify numeric directory names as process IDs.
2. Read /proc/<PID>/stat for process name, state, parent PID, CPU times, thread count, start time, and processor.
3. Read /proc/<PID>/status for VmRSS.
4. Read /proc/<PID>/io for read/write byte counters.
5. Read /proc/stat and /proc/meminfo for system CPU and memory information.
6. Compare current and previous CPU/I/O counters to calculate usage and rates.

7. Sort processes by CPU usage and select either a user-specified PID or the highest-CPU process.
8. Run short strace snapshots for the selected process to collect syscall summaries and recent calls.
9. Update analysis values and alerts, render the terminal dashboard, and repeat approximately every second.

2.4 Advantages of the Proposed Approach

- Uses live Linux kernel information rather than hardcoded process data.
- Monitors multiple processes simultaneously.
- Automatically detects new and terminated processes.
- Provides both system-level and process-level views.
- Allows focused inspection of one selected process.
- Includes CPU, memory, I/O, process state, threads, and syscall activity in one interface.
- Uses threshold persistence of three samples to reduce one-sample alert noise.
- Works as a lightweight terminal application.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement
Processor	Any modern x86-64 processor capable of running Ubuntu/Linux.
RAM	Sufficient memory to run Ubuntu, GCC, strace, and normal background processes.
Storage	Space for source code, object files, executable, and test files.
Display	Terminal/console capable of displaying formatted text and ANSI escape sequences.

3.2 Software Requirements

Software	Purpose
Ubuntu / Linux	Execution environment and access to /proc.
GCC	Compiles the C source files using C11.
GNU Make	Build automation using the supplied Makefile.
strace	Selected-process system-call tracing.
Bash	Running the provided stress-test script.
VS Code / Vim	Source-code editing.

3.3 Operating System Environment — Ubuntu / WSL

The project is designed for Linux/Ubuntu because it directly depends on the Linux /proc virtual filesystem and Linux process information. It can be executed in Ubuntu installed directly on a machine or through WSL when the required Linux packages are available.

3.4 Development Tools and Compilers

The supplied Makefile uses gcc with the options -Wall -Wextra -O2 -std=c11 and links the math library. The project is divided into multiple source modules under src/ and generates object files under build/.

```
CC = gcc
CFLAGS = -Wall -Wextra -O2 -std=c11
LDFLAGS = -lm
make
./kernel_monitor
```

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

4.1 C Programming

The core monitor is implemented in C. C is suitable for this project because it provides direct access to Linux file descriptors, process identifiers, signals, directory traversal, system configuration values, and POSIX APIs.

4.2 GCC Compiler and Make

The project uses GCC with C11. The Makefile automatically compiles every .c file in src/ into build/ object files and links them into the kernel_monitor executable.

4.3 Linux System Calls / APIs

API / Interface	Use in the implementation
fopen(), fgets(), fread(), fclose()	Read Linux virtual files through standard C file I/O.
opendir(), readdir(), closedir()	Scan /proc and /proc/<PID>/task directories.
sysconf(_SC_CLK_TCK)	Obtain clock ticks per second for CPU-time calculations.
clock_gettime()	Measure the monitoring-cycle duration.
sleep()/usleep()	Maintain the approximately one-second refresh interval.
sigaction(), SIGINT, SIGTERM	Stop the monitor safely.
popen()/pclose()	Invoke strace and read its textual output.
fcntl()	Set the strace pipe to non-blocking mode.
kill(), waitpid()	Clean up monitored helper processes when required.

4.4 Linux /proc Filesystem

Path	Information Used
/proc/<PID>/stat	PID, name, state, PPID, utime, stime, thread count, start time, processor.
/proc/<PID>/status	VmRSS for resident memory.
/proc/<PID>/io	read_bytes and write_bytes.
/proc/<PID>/task	Thread enumeration.
/proc/stat	System CPU jiffies and CPU-core count.
/proc/meminfo	MemTotal and MemAvailable.
/proc/uptime	System uptime used for process running-time calculation.

4.5 strace

The project uses strace for selected-process system-call activity. The implementation first checks whether strace is available. For the selected PID, it runs short strace snapshots using -c for syscall

counts and a short traced interval for recent calls. Access may depend on Linux ptrace permissions; the README therefore notes that elevated privileges may be required for some processes.

4.6 Shell Utilities and Testing

The project includes a Bash script named `test_cpu_stress.sh`. It starts CPU-intensive and I/O-intensive test processes so that the monitor can be checked for process discovery, CPU activity, I/O activity, and process removal.

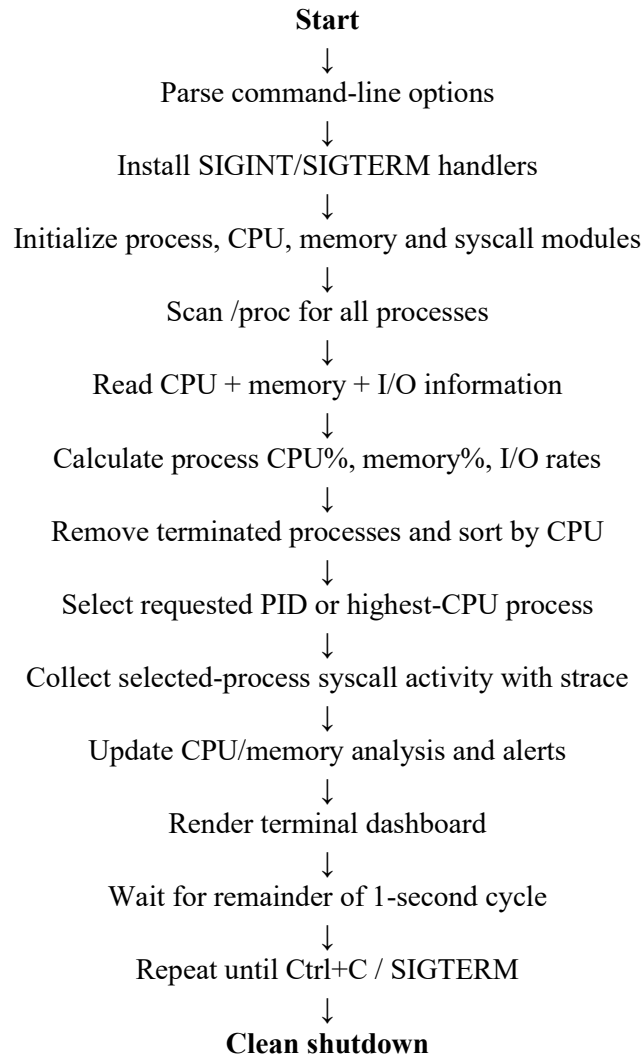
CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Kernel Monitor follows a modular architecture. The main controller coordinates process scanning, CPU and memory collection, I/O calculation, system-call monitoring, alert evaluation, and terminal rendering.

Layer / Module	Responsibility
main.c	Command-line options, signal handling, initialization, monitoring loop, shutdown.
process.c / process.h	Scan /proc, parse process data, track lifecycle, count states, sort processes.
cpu.c / cpu.h	Read system CPU, calculate overall CPU, calculate per-process CPU and analysis.
memory.c / memory.h	Read system memory, calculate per-process memory and analysis.
io.c / io.h	Calculate read/write I/O rates from process counters.
syscall_monitor.c / .h	Use strace to collect syscall counts and recent calls.
alerts.c / alerts.h	Apply CPU/memory thresholds and consecutive-sample alert logic.
display.c / display.h	Render the terminal dashboard.
utils.c / utils.h	Common file-reading, time, uptime, CPU-count, and parsing helpers.

5.2 System Workflow



5.3 Data Flow

The data flow begins with Linux kernel-maintained virtual files and the process table exposed through /proc. The program parses these values into `process_info_t` structures. Current and previous samples are compared to produce CPU percentages and I/O rates. The selected process is passed to the syscall monitor. All processed values are finally passed to the display module.

5.4 Module Description

Module	Key Data / Function
Process Monitoring	PID, name, PPID, state, CPU times, threads, core, running time, lifecycle.
CPU Analysis	Current CPU%, average, peak, minimum, trend, status.
Memory Analysis	VmRSS, memory%, average, peak, minimum, trend, status.
I/O Analysis	Read rate, write rate, I/O status.
Syscall Activity	Syscall counts and recent calls for selected PID.

Module	Key Data / Function
System Analysis	Total/running/sleeping/stopped/zombie counts, overall CPU and memory, health.
Alerts	HIGH/NORMAL process alerts after threshold persistence.

5.5 Input and Output Design

Inputs:

- Optional command-line PID using -p <PID>.
- The live Linux process table and /proc information.
- The current system state and selected process activity.

Outputs:

- Process table sorted by CPU usage.
- Selected-process details.
- CPU, memory and I/O analysis.
- System-call activity.
- Overall system analysis.
- Resource alerts.

CHAPTER 6 — IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is split into separate source files so that process management, resource calculations, system-call tracing, alerts, utilities, and display logic can be developed and tested independently.

Source File	Implementation Details
main.c	Main loop, option parsing, initialization, process selection, rendering, safe termination.
process.c	Reads /proc directories and process files, updates process structures, removes dead processes.
cpu.c	Computes overall and per-process CPU usage from jiffy deltas.
memory.c	Reads MemTotal/MemAvailable and per-process VmRSS; computes percentages.
io.c	Computes MB/s from changes in read_bytes and write_bytes.
syscall_monitor.c	Runs strace snapshots, parses syscall counts and recent syscall names.
alerts.c	Tracks consecutive CPU/memory threshold violations.
display.c	Prints the formatted terminal dashboard.
utils.c	Provides reusable file, time, uptime, CPU-count and conversion helpers.

6.2 Algorithm / Pseudocode

ALGORITHM: Kernel Monitor

1. Parse optional PID argument.
2. Install SIGINT and SIGTERM handlers.
3. Initialize all monitoring structures.
4. Perform an initial /proc scan and CPU read.
5. Repeat while running:
 - a. Scan all numeric PID directories.
 - b. Read /proc/PID/stat, /status and /io.
 - c. Read /proc/stat and /proc/meminfo.
 - d. Calculate CPU and memory usage.
 - e. Calculate I/O rates.
 - f. Remove processes no longer present.
 - g. Sort processes by CPU usage.
 - h. Select user PID, otherwise highest-CPU process.
 - i. Update selected-process CPU/memory analysis.
 - j. Capture selected-process syscall activity with strace.
 - k. Update alerts.
 - l. Render dashboard.
 - m. Wait for the remaining time in the one-second cycle.
6. Stop syscall monitoring and terminate safely.

6.3 Core C Implementation

The process structure `process_info_t` stores the complete set of measurements needed by the monitor. It contains PID information, CPU counters, memory fields, I/O counters, thread information, running time, analysis history, and alert state. A `process_list_t` stores up to 4096 process entries.

```
typedef struct {
    pid_t pid;
    pid_t ppid;
    char name[MAX_NAME];
    char state;
    double cpu_percent;
    double mem_percent;
    double read_rate;
    double write_rate;
    int threads;
    int cpu_core;
    unsigned long running_seconds;
    int cpu_alert;
    int mem_alert;
} process_info_t;
```

6.4 Linux Interfaces / APIs Used

The implementation reads Linux virtual files through standard C file functions. Directory traversal is used for `/proc` and `/proc/<PID>/task`. The program also uses POSIX/Linux APIs for signals, timing, process control, and file descriptor handling.

```
snprintf(path, sizeof(path), "/proc/%d/stat", pid);
read_file_content(path, buf, sizeof(buf));

snprintf(path, sizeof(path), "/proc/%d/status", pid);
read_file_content(path, buf, sizeof(buf));
```

6.5 Process and Thread Monitoring

`process_scan_all()` scans `/proc` and accepts entries whose names begin with a numeric character. For every PID, the monitor parses `/proc/<PID>/stat`, `/status`, and `/io`. If a PID is not already present it is added as a new process. At the end of a cycle, inactive entries are removed. Thread information is obtained from the process stat data and also supported by `/proc/<PID>/task` enumeration.

The process state character is converted to readable labels such as `RUNNING`, `SLEEPING`, `DISK SLEEP`, `STOPPED`, `TRACING`, `ZOMBIE`, `DEAD`, and `IDLE`.

6.6 System Call Monitoring

The selected process is monitored with `strace`. The implementation checks for `strace` using which `strace`. It then uses short tracing intervals to obtain syscall counts and recent syscall names. The design avoids keeping a long blocking trace in the main rendering loop.

```
strace -p <PID> -c -S calls
strace -p <PID> -tt -e trace=all
```

Because strace attaches to a live process, tracing permissions can affect availability. If strace is not installed or attachment is not permitted, syscall information may be unavailable while the remaining monitoring functions continue.

6.7 Memory and I/O Monitoring

Memory monitoring reads VmRSS from `/proc/<PID>/status`. The system memory module reads MemTotal and MemAvailable from `/proc/meminfo`. Process memory percentage is calculated relative to total system memory.

I/O monitoring reads `read_bytes` and `write_bytes` from `/proc/<PID>/io`. The difference between the current and previous counter divided by the monitoring interval gives an approximate MB/s rate.

```
CPU% = (Δprocess_jiffies / Δtotal_jiffies) × 100 × number_of_CPUs  
MEM% = VmRSS / MemTotal × 100  
I/O rate = Δbytes / Δtime
```

6.8 Alert and Signal Handling

The alert module uses an 80% CPU threshold and an 80% memory threshold. A process must remain above the relevant threshold for three consecutive samples before its alert becomes active. If the value falls below the threshold, the corresponding consecutive counter is reset.

SIGINT and SIGTERM are handled by changing the global running flag. The main loop then exits normally, stops syscall monitoring, and prints a safe-shutdown message.

CHAPTER 7 — EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

From the kernel-monitor directory:

```
sudo apt update
sudo apt install build-essential strace make

make
```

The Makefile compiles the source files into build/*.o and links them into the kernel_monitor executable.

7.2 Execution Commands

```
./kernel_monitor
./kernel_monitor -p <PID>
./kernel_monitor -h
```

With no PID argument, the implementation automatically selects the process with the highest measured CPU usage. With -p, the specified PID is used when it is available; if it is not found, the monitor falls back to the highest-CPU process.

7.3 Sample Inputs and Outputs

Example command:

```
./kernel_monitor
```

The implemented display is organized into these sections:

1. Header — process count and refresh information.
2. Process Table — top processes sorted by CPU usage.
3. Selected Process — detailed process information.
4. CPU Analysis — current, average, peak, minimum, trend, and status.
5. Memory Analysis — current, average, peak, minimum, trend, and status.
6. I/O Analysis — read/write rates and status.
7. System Call Activity — syscall counts and recent calls.
8. System Analysis — process counts, overall CPU/memory, and health.
9. Alerts — active threshold alerts.

Actual process names, PIDs, CPU values, memory values, and syscall names depend on the Linux machine at execution time; therefore, this report does not hard-code a fake runtime output.

7.4 Testing

The supplied test_cpu_stress.sh script creates temporary CPU and I/O workloads. It starts a dd process reading random data, a yes process writing continuously to /dev/null, and another dd

process writing to /tmp. The script prints their PIDs and instructs the user to run the monitor in another terminal.

```
cd kernel-monitor
make
./tests/test_cpu_stress.sh

# In another terminal:
./kernel_monitor
```

The intended checks are: the test PIDs should appear in the process table, CPU-intensive processes should show non-zero CPU usage, the I/O-intensive process should show I/O activity, and the test processes should disappear after termination.

7.5 Demonstration of OS Concepts

Demonstration	Observed Through
Process discovery	/proc numeric PID directories.
Process state	/proc/<PID>/stat state field.
CPU usage	jiffy delta calculation.
Memory usage	VmRSS and /proc/meminfo.
I/O activity	/proc/<PID>/io.
Threads	/proc/<PID>/task and process stat.
System calls	strace attached to selected PID.
Signals	Ctrl+C through SIGINT.
Process lifecycle	New PID detection and inactive-process removal.
Resource alerts	80% threshold with 3 consecutive samples.

CHAPTER 8 — CONCLUSION

8.1 Summary of the Project

The Kernel Monitor successfully implements a Linux terminal-based monitoring application in C. It combines live process discovery with CPU, memory, I/O, process-state, thread, system-call, and system-health information. The modular implementation separates data collection, analysis, alerts, and presentation.

8.2 Objectives Achieved

- Real-time CPU and memory monitoring was implemented.
- Multiple running processes are tracked simultaneously.
- Per-process CPU usage is calculated from Linux jiffy data.
- Per-process memory is derived from VmRSS.
- Process I/O rates are calculated from kernel-maintained byte counters.
- Process states, parent PID, thread count, CPU core, and running time are displayed.
- A selected process can be traced for system-call activity using strace.
- New and terminated processes are handled automatically.
- CPU and memory alerts are generated after persistent threshold violations.
- The monitor can be safely stopped with Ctrl+C.

8.3 Results Achieved

The project provides a working terminal dashboard that reflects the live Linux system rather than static sample data. The process list changes as processes start and terminate, CPU and I/O values are updated using consecutive measurements, and selected-process details can be examined separately. The test script provides a repeatable way to generate CPU and I/O activity for demonstration.

8.4 Skills and Learning Outcomes

- Understanding Linux process structures and the /proc filesystem.
- Reading and parsing kernel-provided virtual files in C.
- Calculating CPU utilization from time-based counters.
- Understanding resident memory and process I/O counters.
- Working with POSIX signals and process-related APIs.
- Using strace to observe system-call activity.
- Designing a modular C system-programming project.
- Using Makefiles, GCC, Bash testing, and Ubuntu/WSL.
- Connecting Operating Systems theory with live system behavior.

CHAPTER 9 — REFERENCES

9.1 Books

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, Operating System Concepts.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems.

9.2 Research / Technical Background

- Linux process and resource monitoring concepts as used in the project implementation.
- The project problem statement submitted for Operating Systems and Systems Programming (25CS2104E), 2026–27.

9.3 Linux / GNU Documentation

- Linux /proc filesystem interfaces: /proc/stat, /proc/meminfo, /proc/<PID>/stat, /proc/<PID>/status, /proc/<PID>/io, /proc/<PID>/task.
- GNU GCC compiler and GNU Make documentation.
- Linux/POSIX signal, directory, timing, and file-I/O interfaces.
- strace documentation for Linux system-call tracing.

9.4 Project Files Used

File	Purpose
README.md	Project features, formulas, requirements, usage, architecture, and controls.
Makefile	Build configuration.
src/main.c	Main monitoring loop and program control.
src/process.c/.h	Process discovery and process information.
src/cpu.c/.h	CPU calculations and analysis.
src/memory.c/.h	Memory calculations and analysis.
src/io.c/.h	I/O rate calculation.
src/syscall_monitor.c/.h	strace-based system-call monitoring.
src/display.c/.h	Terminal dashboard rendering.
src/alerts.c/.h	Threshold-based alerts.
src/utlis.c/.h	Common utilities.
tests/test_cpu_stress.sh	CPU/I/O stress testing.

END OF PROJECT DOCUMENTATION